LAPORAN TUGAS PEMROGRAMAN KRIPTOGRAFI

Implementasi Mandiri Protokol Pertukaran Kunci Diffie-Hellman
dan Kriptosistem Asimetris ElGamal Berbasis Logaritma Diskrit

Disusun oleh Kelompok 17 (Kelas A):

Naswan Nashir Ramadhan	(5025231246)
Alfianz Risqia Ilahi Loven Kary	(5025241164)
Az Zahrra Tasya Adelia	(5027241087)

Mata Kuliah Kriptografi (A)
Institut Teknologi Sepuluh Nopember
Semester Ganjil 2026/2027

1 Pendahuluan

1.1 Latar Belakang

Keamanan transmisi data pada saluran komunikasi terbuka mengandalkan dua mekanisme utama, yaitu kesepakatan kunci simetris (*key establishment*) dan enkripsi pesan asimetris. Kriptografi kunci publik bertumpu pada fungsi matematika satu arah terperangkap (*trapdoor one-way function*). Di samping persoalan faktorisasi bilangan bulat besar pada RSA, pilar fundamental lainnya adalah Persoalan Logaritma Diskrit (*Discrete Logarithm Problem* / DLP) pada grup siklik berhingga bilangan bulat modulo prima.

Protokol pertukaran kunci Diffie-Hellman (1976) memungkinkan dua pihak menurunkan kunci sesi simetris bersama tanpa perlu mengirimkan kunci rahasia tersebut melalui jaringan. Melanjutkan fondasi tersebut, Taher ElGamal (1984) memperluas pemanfaatan DLP untuk merancang skema enkripsi dan dekripsi pesan kunci publik yang bersifat probabilistik, di mana enkripsi pesan yang sama dengan kunci publik yang sama akan menghasilkan pasangan teks tersandi (*ciphertext*) yang berbeda karena penyertaan parameter acak pada setiap blok.

1.2 Kriteria dan Batasan Penugasan

Implementasi sistem ini dirancang memenuhi batasan penugasan secara ketat:

- **Bebas Pustaka Kriptografi**: Seluruh operasi matematika teori bilangan dibangun murni dari dasar (*from scratch*) tanpa pustaka pihak ketiga.
- **Modularitas Fungsi**: Setiap tahapan operasional Diffie-Hellman dan ElGamal diisolasi ke dalam fungsi-fungsi independen yang transparan.
- **Verifikasi Materi Kuliah**: Menyediakan skenario pembuktian numerik yang mencocokkan contoh pada materi perkuliahan ke-9 secara identik, meliputi pertukaran kunci Diffie-Hellman ($n = 251, g = 6$) dan enkripsi ElGamal teks “HALO” ($p = 2273, g = 3$).
- **Kelengkapan Pengujian**: Memvalidasi seluruh fungsi menggunakan rangkaian pengujian otomatis (*unit test*) berbasis `pytest`.

2 Landasan Teori dan Formulasi Matematis

2.1 Grup Siklik dan Akar Primitif (Primitive Root)

Misalkan p adalah bilangan prima. Berdasarkan fungsi totient Euler, banyaknya bilangan bulat positif yang relatif prima terhadap p adalah $\varphi(p) = p - 1$. Bilangan bulat g didefinisikan sebagai akar primitif (*generator*) modulo p jika himpunan perpangkatan:

$$\{g^1 \bmod p, \quad g^2 \bmod p, \quad ..., \quad g^{p-1} \bmod p\}$$

menghasilkan permutasi seluruh elemen bilangan bulat tereduksi $\{1, 2, ..., p-1\}$. Artinya, panjang siklus perpangkatan mencapai batas maksimum sepanjang $p - 1$.

Pemeriksaan akar primitif secara efisien tidak memerlukan pengujian ke seluruh $p - 1$ nilai eksponen. Nilai g terbukti sebagai akar primitif sah dari p jika dan hanya jika untuk setiap faktor prima unik q dari $(p - 1)$ berlaku:

$$g^{\frac{p-1}{q}} \neq 1 \pmod p$$

2.2 Persoalan Logaritma Diskrit (Discrete Logarithm Problem)

Diberikan bilangan prima p, basis generator g, dan eksponen privat x, operasi perpangkatan modular:

$$y \equiv g^x \pmod p$$

dapat dihitung secara efisien menggunakan algoritma *Square-and-Multiply* dengan kompleksitas waktu $O(\log x)$.

Sebaliknya, jika hanya diketahui nilai publik y, g, dan p, persoalan untuk memulihkan kembali nilai eksponen rahasia x sedemikian sehingga $g^x \equiv y \pmod{p}$ disebut sebagai *Discrete Logarithm Problem* (DLP). Belum ada algoritma klasik waktu polinomial yang mampu memecahkan persoalan ini untuk nilai prima p yang berukuran besar.

2.3 Protokol Diffie-Hellman Key Exchange (DHKE)

Protokol ini berjalan melalui tahapan aljabar berikut:

- **Parameter Publik**: Alice dan Bob menyepakati bilangan prima n dan generator g. Nilai (n, g) bersifat publik dan aman dipertukarkan di saluran terbuka.
- **Pembangkitan Kunci Privat**: Alice memilih bilangan acak rahasia x ($1 < x < n-1$), sedangkan Bob memilih bilangan acak rahasia y ($1 < y < n-1$).
- **Transmisi Kunci Publik**: Alice menghitung $X = g^x \bmod n$ dan mengirimkannya ke Bob. Bob menghitung $Y = g^y \bmod n$ dan mengirimkannya ke Alice.
- **Penurunan Kunci Sesi Bersama**: Alice menghitung $K = Y^x \bmod n$, sedangkan Bob menghitung $K' = X^y \bmod n$.

Sesuai sifat komutatif eksponensiasi modular:

$$K = (g^y)^x \equiv g^{xy} \equiv (g^x)^y = K' \pmod{n}$$

Kunci sesi simetris $K = K'$ berhasil diperoleh kedua belah pihak tanpa membuka informasi nilai x maupun y kepada penyadap di saluran komunikasi.

2.4 Kriptosistem Kunci Publik ElGamal

Skema ElGamal mencakup tiga prosedur utama:

- **Pembangkitan Kunci**: Penerima memilih prima p, akar primitif g, dan kunci privat acak x ($1 < x < p-1$). Kunci publik dihitung sebagai $y = g^x \bmod p$. Pasangan kunci publik penerima adalah (y, g, p) dan kunci privatnya adalah x.
- **Enkripsi Pesan**: Pesan dipecah menjadi deret blok integer m_i ($0 \leq m_i < p$). Untuk tiap blok, pengirim memilih bilangan acak k ($1 < k < p-1$), lalu menghitung pasangan teks tersandi (a, b):

  $$a = g^k \bmod p$$

  $$b = \left(y^k \cdot m_i\right) \bmod p$$

  Ukuran teks tersandi menjadi dua kali lipat ukuran teks asli karena setiap blok m_i diwakili oleh pasangan nilai (a, b).
- **Dekripsi Pesan**: Penerima memulihkan m_i menggunakan kunci privat x dengan memanfaatkan Teorema Kecil Fermat:

  $$(a^x)^{-1} \equiv a^{p-1-x} \pmod{p}$$

  $$m_i = \left(b \cdot a^{p-1-x}\right) \bmod p$$

3 Struktur Repositori dan Rancang Bangun Sistem

Arsitektur perangkat lunak disusun secara modular dengan memisahkan domain matematika murni, mesin protokol, antarmuka interaktif, dan pengujian otomatis:

Berkas / Modul	Tanggung Jawab dan Fungsi Utama
`crypto_core.py`	Primitif matematika murni independen: uji prima trial division, faktorisasi pembagi totient, verifikasi akar primitif, eksponensiasi Square-and-Multiply, dan invers modular.
`diffie_hellman.py`	Logika protokol pertukaran kunci: validasi parameter publik, kalkulasi kunci publik pihak berkomunikasi, dan derivasi kunci sesi bersama.
`elgamal.py`	Kriptosistem kunci publik ElGamal: pembentukan kunci, partisi blok desimal perkuliahan, partisi biner UTF-8, enkripsi pasangan blok, dan dekripsi modular Fermat.
`app.py`	Antarmuka web interaktif Streamlit: visualisasi saluran Alice-Bob, panel inspeksi penyadap pasif Eve, tabel proses enkripsi/dekripsi per blok, dan lab teori DLP.
`tests/`	Rangkaian 12 pengujian unit otomatis berbasis pytest untuk memverifikasi kebenaran matematis dan kompatibilitas contoh perkuliahan.

4 Penjelasan Fungsi Setiap Tahapan Program

4.1 Modul Primitif Matematika Murni (`crypto_core.py`)

Modul ini menjadi fondasi seluruh kalkulasi aljabar tanpa dependensi eksternal:

- **Uji Keprimaan (`is_prime`)**: Mengimplementasikan *Trial Division* teroptimasi $6k \pm 1$. Setelah memeriksa basis 2 dan 3, perulangan melompati seluruh bilangan genap dan kelipatan 3 hingga batas $\sqrt{n}$, sehingga memangkas kompleksitas komputasi menjadi $O(\sqrt{n})$.
- **Pembagi Bersama Terbesar (`gcd` & `extended_gcd`)**: Menggunakan Algoritma Euclidean untuk memeriksa keterbagian prima relatif, serta Algoritma Euclidean Diperluas untuk menghitung koefisien Bézout ($a \cdot x + b \cdot y = \gcd(a, b)$).
- **Invers Perkalian Modular (`mod_inverse`)**: Menghitung nilai $a^{-1} \bmod m$ yang memenuhi $(a \cdot x) \bmod m = 1$ memanfaatkan koefisien x dari Algoritma Euclidean Diperluas.
- **Eksponensiasi Biner Modular (`mod_pow`)**: Menghitung perpangkatan modular dengan algoritma *Square-and-Multiply*. Eksponen dipindai secara bitwise; basis dikuadratkan pada setiap langkah dan dikalikan dengan basis jika bit aktif bernilai 1. Algoritma ini berjalan dalam kompleksitas waktu $O(\log \exp)$ dan mencegah luapan memori (*memory overflow*).
- **Verifikasi dan Pencarian Akar Primitif (`is_primitive_root` & `find_primitive_root`)**: Fungsi `get_prime_factors()` mengekstrak seluruh faktor prima dari $p - 1$. Fungsi `is_primitive_root()` memvalidasi syarat keterbagian generator, dan fungsi `find_primitive_root()` mencari generator terkecil secara iteratif.

4.2 Tahapan Protokol Diffie-Hellman (`diffie_hellman.py`)

- **Tahap 1: Pembentukan Parameter Publik (`generate_dh_parameters` & `validate_dh_parameters`)**: Membangkitkan bilangan prima acak n dan generator sah g, serta memvalidasi keabsahan parameter sebelum proses pertukaran dimulai.
- **Tahap 2: Pembangkitan Kunci Privat (`generate_dh_private_key`)**: Membangkitkan integer rahasia x dan y secara acak pada rentang $2 \leq x, y \leq n - 2$.

- **Tahap 3: Kalkulasi Kunci Publik (`compute_dh_public_key`)**: Mengeksekusi eksponensiasi modular publik $X = g^x \bmod n$ dan $Y = g^y \bmod n$ menggunakan fungsi `mod_pow()`.
- **Tahap 4: Penurunan Kunci Sesi Bersama (`compute_dh_shared_secret`)**: Menurunkan kunci sesi menggunakan kunci publik pihak lawan dan kunci privat sendiri: $K = Y^x \bmod n$ dan $K' = X^y \bmod n$.

4.3 Tahapan Kriptosistem ElGamal (`elgamal.py`)

- **Tahap 1: Pembangkitan Kunci (`generate_elgamal_keypair`)**: Menerima parameter prima p dan generator g. Menghasilkan kunci privat x dan kunci publik $y = g^x \bmod p$.
- **Tahap 2: Pemecahan Blok Pesan (`text_to_lecture_blocks` & `text_to_utf8_blocks`)**:
  - Mode Kuliah: Memetakan alfabet ke dua digit angka ($A = 00 ... Z = 25$) lalu membaginya ke dalam blok 4-digit agar nilai $m_i < p$.
  - Mode UTF-8: Mengonversi teks ke deret biner lalu mempartisinya ke dalam integer Big-Endian berukuran aman terhadap panjang bit modulus p.
- **Tahap 3: Enkripsi Blok Pesan (`encrypt_elgamal_blocks`)**: Untuk setiap blok pesan m_i, dipilih nilai acak k, lalu dihitung $a = g^k \bmod p$ dan $b = \left(y^k \cdot m_i\right) \bmod p$.
- **Tahap 4: Dekripsi Blok Pesan (`decrypt_elgamal_blocks`)**: Memulihkan integer pesan m_i via invers Teorema Kecil Fermat menggunakan eksponen $p - 1 - x$.
- **Tahap 5: Rekonstruksi Teks dan Pelepasan Padding (`lecture_blocks_to_text` & `utf8_blocks_to_text`)**: Mengonversi kembali integer pesan m_i ke teks asli dan melepaskan karakter pengganjal (*padding*) secara presisi.

4.4 Penanganan Blok Ganjil dan Padding Deterministik

Pada mode perkuliahan, pemetaan dua digit menghasilkan blok 4-digit (setara dua huruf per blok). Apabila teks masukan memiliki panjang ganjil (seperti kata "LOGARITMA" yang terdiri atas 9 huruf), huruf terakhir tidak dapat membentuk pasangan 4-digit secara utuh.

Sistem menerapkan penanganan bantalan (*padding*) berintegritas:

- Enkoder menyisipkan satu huruf pengganjal 'X' di akhir deretan sehingga panjang karakter genap (menjadi "LOGARITMAX" / 5 blok) dan menyimpan jumlah bantalan (`pad_count = 1`).
- Pada tahap dekripsi, setelah kelima blok dipulihkan secara matematis menjadi deretan karakter, fungsi pemulihan memotong sejumlah `pad_count` karakter dari posisi akhir. Teks kembali pulih menjadi "LOGARITMA", dan antarmuka memberikan catatan transparan mengenai pelepasan karakter pengganjal tersebut.

5 Dokumentasi Antarmuka Web Interaktif (`app.py`)

Dashboard interaktif dibangun menggunakan Streamlit dengan tata letak visual multi-tab:

5.1 Tab 1: Pertukaran Kunci Diffie-Hellman

- Menyediakan tombol preset materi perkuliahan ($n = 251, g = 6, x = 124, y = 158$) untuk demonstrasi instan, serta tombol acak parameter baru.
- Memisahkan visualisasi komputasi Sisi Alice dan Sisi Bob secara berdampingan.
- Menyediakan panel edukasi sudut pandang penyadap (Eve) yang menginspeksi nilai apa saja yang bocor ke saluran publik (n, g, X, Y) dan menjelaskan mengapa Eve tidak dapat memulihkan kunci sesi K tanpa memecahkan persoalan logaritma diskrit.

5.2 Tab 2: Kriptosistem ElGamal

- Mendukung dua skema pengodean: Mode Materi Kuliah ($A = 00 ... Z = 25$) dan Mode Teks Umum (UTF-8) untuk teks panjang, huruf kecil, dan simbol.

- Menyediakan tabel detail per blok yang menampilkan nilai integer m, nilai acak k, $a = g^k \bmod p$, $b = \left(y^k \cdot m\right) \bmod p$, dan pasangan teks tersandi (a, b).
- Menampilkan tabel proses dekripsi per blok yang memverifikasi kalkulasi $a^{p-1-x} \bmod p$ hingga pesan asli pulih seutuhnya.
- Menggunakan mekanisme fungsi alih status (*callback*) yang secara otomatis membersihkan tabel komputasi lama saat pengguna beralih mode.

5.3 Tab 3: Laboratorium Teori Akar Primitif & DLP

- **Uji Siklus Generator**: Menghitung dan menampilkan tabel perpangkatan $g^i \bmod p$ untuk membuktikan apakah nilai g membentuk $p - 1$ elemen unik (akar primitif) atau mengalami siklus berulang sebelum mencapai $p - 1$.
- **Simulasi DLP**: Menyediakan penelusuran linier (*brute-force search*) untuk mencari nilai eksponen rahasia x pada kongruensi $b \equiv g^x \pmod p$, mendemonstrasikan kompleksitas komputasi pencarian logaritma diskrit.

6 Pengujian dan Pembuktian Numerik

6.1 Verifikasi Diffie-Hellman Sesuai Slide Perkuliahan

Pengujian mencocokkan perhitungan contoh slide pertemuan ke-9:

- Parameter Publik: $n = 251$ (prima), $g = 6$ (akar primitif sah dengan panjang siklus 250).
- Kunci Privat: Alice $x = 124$, Bob $y = 158$.
- Kunci Publik yang Ditransmisikan:

$$X = 6^{124} \bmod 251 = 209$$

$$Y = 6^{158} \bmod 251 = 197$$

- Derivasi Kunci Sesi Bersama:

$$K_{\text{Alice}} = 197^{124} \bmod 251 = 79$$

$$K'_{\text{Bob}} = 209^{158} \bmod 251 = 79$$

Hasil komputasi program membuktikan $K = K' = 79$ secara identik.

6.2 Verifikasi ElGamal Teks ‘HALO’ Sesuai Slide Perkuliahan

Pengujian mencocokkan parameter enkripsi teks “HALO”:

- Parameter Kunci: Prima $p = 2273$, generator $g = 3$, kunci privat penerima $x = 243$.
- Kunci Publik Penerima:

$$y = 3^{243} \bmod 2273 = 461$$

- Teks “HALO” dipetakan menjadi deret angka 07001114, lalu dipartisi menjadi dua blok 4-digit: $m_1 = 700$ dan $m_2 = 1114$.
- Enkripsi Blok ($k_1 = 1463, k_2 = 2001$):
  - Blok 1: $a_1 = 3^{1463} \bmod 2273 = 1439$, $b_1 = \left(461^{1463} \cdot 700\right) \bmod 2273 = 74$. Diperoleh pasangan $c_1 = (1439, 74)$.
  - Blok 2: $a_2 = 3^{2001} \bmod 2273 = 1220$, $b_2 = \left(461^{2001} \cdot 1114\right) \bmod 2273 = 1682$. Diperoleh pasangan $c_2 = (1220, 1682)$.
- Dekripsi via Teorema Kecil Fermat ($p - 1 - x = 2273 - 1 - 243 = 2029$):
  - Blok 1: $a_1^{2029} = 1439^{2029} \bmod 2273 = 1791$, $m_1 = (74 \cdot 1791) \bmod 2273 = 700$ (“0700” $\rightarrow$ “HA”).

- Blok 2: $a_2^{2029} = 1220^{2029} \bmod 2273 = 1125$, $m_2 = (1682 \cdot 1125) \bmod 2273 = 1114$ (“1114” $\rightarrow$ “LO”).

Teks pulih sempurna menjadi “HALO”.

6.3 Hasil Uji Otomatis Unit Test

Rangkaian pengujian unit test dijalankan melalui framework `pytest` pada lingkungan CachyOS (Python 3.14). Seluruh 12 skenario pengujian berstatus **PASSED**:

Nama Test Case	Fokus Pengujian	Status
`test_is_prime`	Uji batas bilangan bulat negatif, 0, 1, prima kecil, dan komposit	PASSED
`test_gcd_and_extended_gcd`	Kebenaran pembagi terbesar dan relasi Bézout a*x + b*y = gcd(a, b)	PASSED
`test_mod_inverse`	Eksistensi invers perkalian modular via Extended Euclidean	PASSED
`test_mod_pow`	Akurasi eksponensiasi biner modular Square-and-Multiply	PASSED
`test_primitive_root_validation`	Validasi generator sah g=3 vs non-generator g=2 modulo 7	PASSED
`test_find_primitive_root`	Pencarian otomatis akar primitif terkecil sebuah bilangan prima	PASSED
`test_generate_prime`	Pembangkitan bilangan prima acak dalam rentang nilai tertentu	PASSED
`test_dh_slide_example`	Validasi kesesuaian numerik Diffie-Hellman contoh perkuliahan	PASSED
`test_dh_random_flow`	Konsistensi derivasi kunci sesi bersama pada parameter acak	PASSED
`test_elgamal_slide_halo_example`	Validasi kesesuaian numerik ElGamal teks ‘HALO’ contoh kuliah	PASSED
`test_elgamal_lecture_padding`	Integritas penanganan padding ‘X’ pada teks ganjil ‘LOGARITMA’	PASSED
`test_elgamal_utf8_mode`	Enkripsi dan dekripsi teks umum UTF-8 dengan spasi dan simbol	PASSED

7 Kesimpulan

Sistem kriptografi Diffie-Hellman dan ElGamal telah berhasil diimplementasikan secara mandiri dari dasar dengan memenuhi seluruh kriteria penugasan:

- Seluruh algoritma teori bilangan dibangun secara transparan pada modul `crypto_core.py` murni tanpa ketergantungan pada pustaka kriptografi eksternal.
- Komputasi aljabar modular program terbukti akurat dan identik terhadap contoh numerik perkuliahan pada slide pertemuan ke-9.

- Sistem mendukung skema perkuliahan (pemetaan desimal dua digit dengan pelepasan bantalan otomatis) serta skema biner standar industri (UTF-8).
- Antarmuka Streamlit menyediakan visualisasi transparan untuk setiap tahapan pertukaran kunci, proses enkripsi/dekripsi per blok, serta inspeksi keamanan terhadap penyadap pasif.